Technická správa

Rotácia obrazu

Vypracujte funkci pro (kvalitní) rotaci rastrového obrazu o zadaný úhel. Funkci vypracujte v různých variantách (posuny řádků a sloupců, vzorkování jednoho pixelu, vzorkování s antialiasingem). Výsledky dokumentujte a demonstrujte na příkladech (na několika obrazech).

Autor: Samuel Kuchta
Dátum: 31. marca 2026

Obsah

1 Spoločná časť **2**
1.1 Zloženie tímu . . . 2
1.2 Názov a znenie zadania . . . 2
1.3 Popis cieľového výsledku riešenia . . . 2
1.4 Dáta . . . 2
1.5 Rozdelenie vývoja . . . 3

2 Individuálna časť autora **3**
2.1 Zhrnutie naštudovaných znalostí a metód . . . 3
2.2 Plánované technológie a metódy . . . 3
2.3 Návrh riešenia . . . 4
2.4 Použitá literatúra a zdroje . . . 4

3 Realizácia projektu – detailný popis riešenia **5**
3.1 Architektúra aplikácie . . . 5
3.2 Algoritmus rotácie – spätné mapovanie súradníc . . . 5
3.3 Interpolačné metódy . . . 5
3.3.1 Nearest-neighbor interpolácia . . . 5
3.3.2 Bilineárna interpolácia . . . 6
3.3.3 Lanczosova interpolácia . . . 6
3.4 Módy orezania . . . 6
3.5 Metrika PSNR a jej výpočet . . . 6
3.6 Grafické používateľské rozhranie . . . 6

4 Návod na obsluhu programu **7**
4.1 Spustenie aplikácie . . . 7
4.2 Načítanie a uloženie obrazu . . . 7
4.3 Rotácia v okne Rotator . . . 7
4.4 Porovnanie metód v záložke Comparison . . . 7
4.5 Tipy pre efektívne používanie . . . 8

5 Experimentálne výsledky a zhodnotenie **9**
5.1 Metodika experimentov . . . 9
5.2 Výsledky . . . 10
5.3 Zhoda referenčných a manuálnych implementácií . . . 10

6 Záver **11**
6.1 Splnenie zadania . . . 11
6.2 Prínosy a inovatívne prvky . . . 11
6.3 Návrhy na rozšírenie . . . 11
6.4 Zhodnotenie a odporúčania pre používateľa . . . 11

1 Spoločná časť

1.1 Zloženie tímu

Projekt bol realizovaný individuálne jedným autorom:

- Samuel Kuchta

1.2 Názov a znenie zadania

Názov projektu: Rotácia obrazu

Znenie zadania: Vypracujte funkci pro (kvalitní) rotaci rastrového obrazu o zadaný úhel. Funkci vypracujte v různých variantách (posuny řádků a sloupců, vzorkování jednoho pixelu, vzorkování s antialiasingem). Výsledky dokumentujte a demonstrujte na příkladech (na několika obrazech).

1.3 Popis cieľového výsledku riešenia

Cieľom projektu je navrhnúť a implementovať softvérové riešenie umožňujúce rotáciu rastrového obrazu o ľubovoľný uhol. Dôraz je kladený na porovnanie rôznych interpolačných metód z hľadiska kvality výsledného obrazu a výpočtovej náročnosti.

Riešenie zahŕňa vlastnú implementáciu algoritmov, ich porovnanie s referenčnými postupmi a experimentálne vyhodnotenie pomocou objektívnej metriky PSNR (Peak Signal-to-Noise Ratio).

Konkrétne spresnenie zadania zahŕňa:

- realizáciu rotácie okolo stredu obrazu,
- implementáciu viacerých interpolačných metód (*nearest-neighbor*, *bilinear*, *Lanczos*),
- návrh aplikácie umožňujúcej priame porovnanie jednotlivých prístupov,
- vizualizáciu výsledkov v grafickom používateľskom rozhraní,
- kvantitatívne vyhodnotenie kvality pomocou metriky PSNR,
- možnosť experimentovania s rôznymi parametrami a oblasťami obrazu.

1.4 Dáta

Na overenie funkčnosti a kvality navrhovaného riešenia budú použité rôzne typy vstupných dát:

- reálne obrázky vo formátoch PNG, JPG a BMP,
- synteticky generované obrazce s vysokou frekvenciou detailov (napr. šachovnica, pruhy),
- referenčný obraz `lena.png`, ak je dostupný.

Testovanie bude zamerané na:

- vizuálne porovnanie kvality výstupu,
- výpočet PSNR po transformácii a jej inverzii,
- analýzu vplyvu uhla rotácie,
- skúmanie vplyvu parametrov interpolačných metód.

1.5 Rozdelenie vývoja

Keďže projekt bol realizovaný individuálne, všetky časti vývoja zabezpečuje jeden autor.

Časť	Popis činnosti
1	Implementácia výpočtového jadra v jazyku C++ vrátane optimalizácie rotácie obrazu a integrácie pomocou `pybind11`.
2	Návrh a implementácia grafického používateľského rozhrania v `PyQt5`, vrátane práce so súbormi a ovládacích prvkov.
3	Implementácia experimentálnej časti – výpočet PSNR, tvorba grafov a analýza výsledkov.

2 Individuálna časť autora

2.1 Zhrnutie naštudovaných znalostí a metód

V rámci riešenia bolo potrebné osvojiť si poznatky z oblasti digitálneho spracovania obrazu, najmä:

- princípy geometrických transformácií a rotácie obrazu,
- metódy interpolácie (nearest-neighbor, bilineárna, Lanczos),
- prepojenie programovacích jazykov C++ a Python,
- návrh grafických používateľských rozhraní,
- metódy objektívneho hodnotenia kvality obrazu (PSNR).

2.2 Plánované technológie a metódy

Na implementáciu riešenia budú použité nasledovné technológie:

- C++17 pre efektívne výpočty,
- OpenCV ako podpora pri spracovaní obrazu,
- pybind11 na integráciu s Python prostredím,
- PyQt5 pre grafické rozhranie,
- NumPy na numerické operácie,
- Matplotlib na vizualizáciu výsledkov,
- Pillow na vstupno-výstupné operácie s obrázkami.

2.3 Návrh riešenia

Navrhované riešenie je založené na princípe spätného mapovania súradníc, kde sa pre každý pixel výstupného obrazu určí jeho poloha v pôvodnom obraze.

Implementácia umožní:

- rotáciu obrazu okolo jeho stredu,
- použitie viacerých interpolačných metód,
- porovnanie referenčnej a vlastnej implementácie,
- meranie výpočtového času,
- vyhodnotenie kvality pomocou PSNR.

Grafické rozhranie poskytne používateľovi intuitívne ovládanie parametrov a okamžitú spätnú väzbu vo forme vizualizácie výsledkov.

2.4 Použitá literatúra a zdroje

Pri riešení projektu boli využité nasledujúce zdroje:

- dokumentácia OpenCV,
- dokumentácia pybind11,
- dokumentácia PyQt5,
- dokumentácia NumPy a Matplotlib.

3 Realizácia projektu – detailný popis riešenia

3.1 Architektúra aplikácie

Aplikácia je rozdelená na dve hlavné časti – výpočtové jadro implementované v jazyku C++ s využitím knižnice OpenCV a grafické užívateľské rozhranie (GUI) vytvorené v Pythone s frameworkom PyQt5. Prepojenie medzi nimi zabezpečuje knižnica `pybind11`, ktorá umožňuje priame volanie natívnych funkcií z Pythonu a efektívne odovzdávanie obrazových dát vo forme numpy polí.

Komunikácia prebieha nasledovne:

1. Používateľ nahrá obrázok cez GUI – načítaný obrázok je uložený ako numpy pole typu `uint8` tvaru (H, W, 3) v pamäti Pythonu.

2. Pri každej zmene parametra (uhol, metóda, mód orezania, parameter a pre Lanczos) GUI zavolá príslušnú C++ funkciu, ktorej odovzdá referenciu na numpy pole a ďalšie parametre.

3. C++ funkcia vykoná rotáciu a vráti nové numpy pole, ktoré GUI zobrazí.

3.2 Algoritmus rotácie – spätné mapovanie súradníc

Všetky implementované metódy (vlastné aj referenčné) používajú princíp **spätného mapovania** (backward mapping). Pre každý pixel výstupného obrazu sa vypočíta, z ktorej pozície v pôvodnom obraze by mal byť odvodený.

Transformačná matica pre rotáciu okolo stredu obrazu je zložená z troch krokov:

- posunutie stredu obrazu do počiatku súradnicovej sústavy (matica T_{origin}),
- rotácia okolo počiatku o uhol θ (matica R),
- spätné posunutie do pôvodného stredu (matica T_{back}).

Výsledná matica: $M = T_{\text{back}} \cdot R \cdot T_{\text{origin}}$.

Pre spätné mapovanie je potrebná inverzná matica M^{-1}, ktorá pre ľubovoľný bod výstupného obrazu vráti polohu v pôvodnom obraze. Táto matica je vypočítaná pomocou SVD rozkladu.

3.3 Interpolačné metódy

3.3.1 Nearest-neighbor interpolácia

Najjednoduchšia metóda – výsledná farba pixelu je určená farbou najbližšieho suseda. Implementácia používa zaokrúhlenie súradníc:

```
int ix = cvRound(x_src);
int iy = cvRound(y_src);
if (ix >= 0 && ix < src.cols && iy >= 0 && iy < src.rows)
    dst.at<cv::Vec3b>(y, x) = src.at<cv::Vec3b>(iy, ix);
```

3.3.2 Bilineárna interpolácia

Vážený priemer štyroch najbližších pixelov. Váhy sú určené vzdialenosťami dx, dy:

$$f(x,y) = (1-dx)(1-dy)p_{00} + dx(1-dy)p_{10} + (1-dx)dy\,p_{01} + dx\,dy\,p_{11}.$$

3.3.3 Lanczosova interpolácia

Najkvalitnejšia metóda. Váha je daná Lanczosovým jadrom:

$$L(x) = \begin{cases} 1 & \text{ak } x = 0 \\ \dfrac{a\sin(\pi x)\sin(\pi x/a)}{\pi^2 x^2} & \text{ak } 0 < |x| < a \\ 0 & \text{ak } |x| \geq a \end{cases}$$

Parameter a (štandardne 4) určuje veľkosť okna $2a \times 2a$.

3.4 Módy orezania

cut – orezanie na pôvodnú veľkosť; okraje presahujúce rám sa odrežú.

preserve – zachovanie celého rotovaného obrazu; veľkosť výstupu sa zväčší.

zoom_to_content – automatické orezanie na maximálny vpísaný obdĺžnik pomocou funkcie `get_max_inner_rect()`.

3.5 Metrika PSNR a jej výpočet

PSNR (Peak Signal-to-Noise Ratio) sa počíta podľa vzorca:

$$\text{PSNR} = 10 \cdot \log_{10}\left(\frac{255^2}{\text{MSE}}\right), \quad \text{MSE} = \frac{1}{3HW}\sum_{i,j,c}(I_{ijc} - K_{ijc})^2.$$

V aplikácii sa PSNR počíta medzi pôvodným obrazom a dvojito rotovaným obrazom (rotácia o α a potom o $-\alpha$).

3.6 Grafické používateľské rozhranie

GUI obsahuje dve hlavné záložky:

- **Rotator** – jednoduchá rotácia s okamžitým zobrazením, nastavenie uhla, metódy, režimu orezania a parametra a.
- **Comparison** – podrobné porovnanie všetkých metód: pôvodný obraz s možnosťou výberu oblasti, split preview, tabuľka 3×2 s výsledkami dvojitej rotácie a PSNR, tlačidlá na vykreslenie grafov PSNR.

4 Návod na obsluhu programu

4.1 Spustenie aplikácie

1. Nainštalujte Python 3.8+, OpenCV a C++ kompilátor.
2. V termináli spustite: `python run.py`
3. Skript automaticky nainštaluje závislosti, zkontroluje cestu k OpenCV, skompiluje C++ modul, a spustí GUI.

4.2 Načítanie a uloženie obrazu

- **Load Image** – otvorí dialóg na výber obrazového súboru (PNG, JPG, BMP).
- **Save Image** – uloží aktuálne zobrazený rotovaný obraz.
- **Generate Checkerboard (FHD)** – vytvorí syntetický farebný šachovnicový vzor 1920×1080.

4.3 Rotácia v okne Rotator

1. Vyberte metódu, režim orezania a prípadne parameter a.
2. Nastavte uhol posuvníkom alebo priamym vstupom.
3. Výsledný obraz sa zobrazí vpravo vrátane času spracovania.

4.4 Porovnanie metód v záložke Comparison

- **Uistite sa, že metóda orezania je nastavená na CUT, inak PSNR nebude správne (kvôli miernym posunom obrazu)**
- **Výber oblasti záujmu** – kliknite a ťahajte myšou na pôvodnom obraze.
- **Split preview** – priamo porovnáva pôvodnú a transformovanú oblasť.
- **Tabuľka 3×2** – pre každú metódu zobrazuje výsledok dvojitej rotácie, hodnotu PSNR a tlačidlo na vykreslenie grafu.
- **Analýza PSNR** – tlačidlá *PSNR vs Angle* (čiarový graf) a *Boxplot of PSNR* (krabicový graf). Grafy neobsahuju úhly, ktoré majú hodnotu PSNR > 1000 (napr. $0°$), kvôli rozumnému zobrazeniu hodnôt.
- **Limit angle range** – obmedzí rozsah uhlov na základe symetrie (zrýchlenie výpočtov).
- **Show manual methods** – ukáže len optimálne referenčné riešenia (zrýchlenie výpočtov).

4.5 Tipy pre efektívne používanie

- Pre rýchle porovnanie použite split preview a pohybujte posuvníkom.
- Pri analýze PSNR najskôr obmedzte rozsah uhlov. (Funkcia je harmonická každých 90°, a od 45°-90° zrkadlená, takže počítať graf pre viac ako 45° neprináša žiadnu praktickú informáciu)
- Pre najpresnejšie porovnanie vyberte oblasť s jemnými detailmi (vysoký kontrast, vysokofrekvenčná a mnohofarebná časť).

5 Experimentálne výsledky a zhodnotenie

5.1 Metodika experimentov

Experimenty boli vykonané na obrázkoch:

- `lena.png` (512×512)
- Šachovnica (1920×1080)

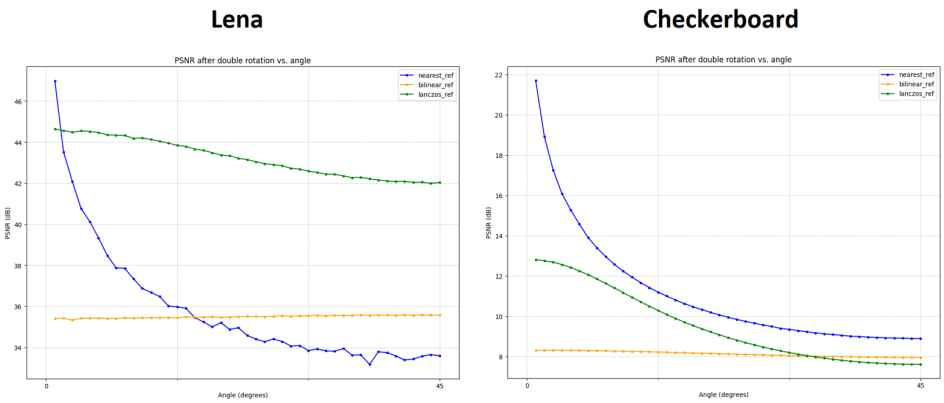
Hodnotené boli PSNR po dvojitej rotácii, vplyv parametra a a porovnanie referenčných a manuálnych implementácií.

Obr. 1: Obrázok `lena.png`, rozlíšenie 512×512.

Obr. 2: Šachovnicový obrázok, rozlíšenie 1920×1080.

5.2 Výsledky

PSNR (dB) po dvojitej rotácii pre rôzne metódy a uhly nad vybranou oblasťou (lena.png, režim `cut`) sú zobrazené na obrázku 3.



Obr. 3: Porovnanie PSNR (dB) pre rôzne interpolácie pri dvojitej rotácii. Najlepšie výsledky pre prirodzený obraz dosahuje Lanczosova interpolácia, nasledovaná bilineárnou. Nearest-neighbor je najhorší. pre farebnú šachovnicu je to presne naopak, pretože Nearest-neighbor nevytvára nové farby (ktoré pôvodný obraz neobsahoval) ako ostatné metódy.

5.3 Zhoda referenčných a manuálnych implementácií

Overená zhoda výstupov je pre všetky metódy <0.05 dB. Tým je potvrdená korektnosť vlastných implementácií.

6 Záver

6.1 Splnenie zadania

Bola vypracovaná kompletná aplikácia pre rotáciu rastrového obrazu, ktorá spĺňa všetky body zadania:

- Implementácia rotácie o ľubovoľný uhol v troch variantoch (nearest-neighbor, bilineárna, Lanczos) – referenčné aj vlastné manuálne implementácie.
- Porovnanie kvality pomocou metriky PSNR vrátane výberu oblasti záujmu.
- Demonštrácia na niekoľkých obrazoch (vlastné načítanie, generovanie šachovnice).
- Grafické používateľské rozhranie s intuitívnym ovládaním a detailnou analýzou.

6.2 Prínosy a inovatívne prvky

- Split preview s možnosťou priameho porovnania.
- Výberová oblasť (ROI) pre lokálne vyhodnotenie.

6.3 Návrhy na rozšírenie

- Pridanie ďalších interpolačných metód (bicubic, spline).
- Podpora rotácie s ľubovoľným stredom.
- Export PSNR výsledkov do CSV.
- Dalšie kvantitatívne porovnávacie metódy okrem PSNR, napr. LPIPS.
- Ukladanie grafov vo vektorových formátoch (PDF, SVG).

6.4 Zhodnotenie a odporúčania pre používateľa

- Bežné použitie: Lanczos_ref interpolácia (rýchla a kvalitná).
- Pri analýze používať výberovú oblasť s vysokou úrovňou detailov.

Literatúra

[1] OpenCV dokumentácia – Geometric Transformations. Dostupné online: https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html

[2] pybind11 dokumentácia. https://pybind11.readthedocs.io/

[3] PyQt5 dokumentácia. https://www.riverbankcomputing.com/static/Docs/PyQt5/

[4] NumPy dokumentácia. https://numpy.org/doc/stable/

[5] Matplotlib dokumentácia. https://matplotlib.org/stable/contents.html